OS Lab

Skill-19

To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t foreground_pid = -1;

/* Handle SIGINT received by the shell */
void handle_sigint(int sig)
{
printf("\nShell received SIGINT\n");

if (foreground_pid > 0)
{
printf(
"Forwarding SIGINT to foreground process %d\n",
foreground_pid
);

kill(foreground_pid, SIGINT);
}
else
{
printf("No foreground process running\n");
}

printf("signal$ ");
fflush(stdout);
}

/* Handle SIGCHLD when a child changes state */
```

```
void handle_sigchld(int sig)
{
/*
* The shell does not terminate when
* the child process terminates.
*/
printf("\nForeground process state changed\n");
}

/* Install signal handlers */
void setup_signal_handlers()
{
struct sigaction sa;

memset(
&sa,
0,
sizeof(sa)
);

sa.sa_handler = handle_sigint;

sigemptyset(
&sa.sa_mask
);

sa.sa_flags = 0;

if (sigaction(
SIGINT,
&sa,
NULL
) == -1)
{
perror("sigaction SIGINT");
exit(EXIT_FAILURE);
}

/*
* Register SIGCHLD handler.
*/
memset(
&sa,
0,
```

```
sizeof(sa)
);

sa.sa_handler = handle_sigchld;

sigemptyset(
&sa.sa_mask
);

sa.sa_flags = 0;

if (sigaction(
SIGCHLD,
&sa,
NULL
) == -1)
{
perror("sigaction SIGCHLD");
exit(EXIT_FAILURE);
}
}

/* Execute foreground command */
void execute_command(char *input)
{
char input_copy[100];

strcpy(
input_copy,
input
);

char *args[20];
int argc = 0;

char *token = strtok(
input,
" \t\n"
);

while (
token != NULL &&
argc < 19
)
```

```
{
args[argc++] = token;

token = strtok(
NULL,
" \t\n"
);
}

args[argc] = NULL;

if (argc == 0)
{
return;
}

pid_t pid = fork();

if (pid < 0)
{
perror("fork");
return;
}

if (pid == 0)
{
/*
* Child gets default SIGINT behavior.
* Therefore Ctrl+C can terminate it.
*/
signal(
SIGINT,
SIG_DFL
);

signal(
SIGCHLD,
SIG_DFL
);

execvp(
args[0],
args
);
```

```
perror("execvp");
exit(EXIT_FAILURE);
}

/*
* Parent records the foreground process.
*/
foreground_pid = pid;

printf(
"Started foreground process PID=%d\n",
pid
);

int status;

/*
* Wait for the foreground process.
*/
waitpid(
pid,
&status,
0
);

/*
* Clear foreground process.
*/
foreground_pid = -1;

/*
* Update job state.
*/
if (WIFEXITED(status))
{
printf(
"\nForeground process completed with status %d\n",
WEXITSTATUS(status)
);
}
else if (WIFSIGNALED(status))
{
printf(
```

```
"\nForeground process terminated by signal %d\n",
WTERMSIG(status)
);
}

printf("\n");
}

/* Main shell */
int main()
{
char input[100];

/*
* Register signal handlers.
*/
setup_signal_handlers();

printf("=== Signal Handling Demo ===\n");
printf("Ctrl+C sends SIGINT.\n");
printf("The shell forwards SIGINT to the foreground process.\n");
printf("The shell itself remains running.\n");
printf("Type 'exit' to quit.\n\n");

while (1)
{
printf("signal$ ");
fflush(stdout);

if (fgets(
input,
sizeof(input),
stdin
) == NULL)
{
break;
}

input[
strcspn(input, "\n")
] = '\0';

if (strlen(input) == 0)
{
```

```
continue;
}

if (strcmp(
input,
"exit"
) == 0)
{
break;
}

execute_command(input);
}

printf("Shell exiting...\n");

return 0;
}
```

```
lahari@DESKTOP-AIC33F7:~$ cd /home/coder/project
nano signal_handling.c
-bash: cd: /home/coder/project: No such file or directory
lahari@DESKTOP-AIC33F7:~$ gcc signal_handling.c -o signal_handling
lahari@DESKTOP-AIC33F7:~$ ./signal_handling
=== Signal Handling Demo ===
Ctrl+C sends SIGINT.
The shell forwards SIGINT to the foreground process.
The shell itself remains running.
Type 'exit' to quit.

signal$ sleep 30
Started foreground process PID=492
^C
Shell received SIGINT
Forwarding SIGINT to foreground process 492
signal$
Foreground process completed with status 0

signal$
Foreground process state changed
Shell exiting...
lahari@DESKTOP-AIC33F7:~$
```